

第一部分：理论篇

1.1 什么是 RAID？

RAID 全称是 Redundant Array of Independent Disks，中文叫做**独立磁盘冗余阵列**。

简单来说，RAID 就是把多块硬盘组合起来，当作一块硬盘来使用的技术。

生活化类比：搬砖的工人

想象你要把 1000 块砖从 A 地搬到 B 地：

方式	类比	效果
1 个工人搬	1 块硬盘	速度慢，工人累倒就完了
4 个工人一起搬	4 块硬盘组 RAID	速度快，一个工人累倒其他人能顶上

这就是 RAID 的核心思想：**速度更快、更安全**。

1.2 为什么需要 RAID？

RAID 解决三个核心问题：

第一，容量问题。单块硬盘容量有限，多块硬盘组合起来容量更大。就像一个钱包装不下所有钱，那就多买几个钱包。

第二，性能问题。多块硬盘同时读写，速度翻倍。就像收银台只有一个时排长队，开四个收银台就快多了。

第三，安全问题。硬盘会坏，数据会丢。RAID 可以让一块硬盘坏了数据也不丢。就像重要文件你会多复印几份放在不同地方。

1.3 RAID 级别详解

RAID 0：追求速度（条带化）

工作原理：数据被分割成块，交替写入多块硬盘。

生活类比：两个人一起抄作业，你抄奇数页我抄偶数页，速度翻倍，但任何一个人的纸丢了作业就不完整了。

特性	说明
最少硬盘数	2 块
可用容量	所有硬盘容量之和（100%）
读写性能	翻倍提升
容错能力	无（任意一块盘坏，数据全丢）
适用场景	临时数据、视频剪辑缓存

RAID 1：追求安全（镜像）

工作原理：数据同时写入两块硬盘，完全一样的两份。

生活类比：重要证件复印两份，一份放家里一份放办公室，丢了一份还有另一份。

特性	说明
最少硬盘数	2 块
可用容量	总容量的 50%
读写性能	读取可提升，写入无提升
容错能力	可坏 1 块盘
适用场景	系统盘、重要数据库

RAID 5：速度与安全的平衡

工作原理：数据和校验信息分散存储在所有硬盘上，任意坏一块盘可通过计算恢复数据。

生活类比：考试时三个同学 A、B、C 互相记对方的答案。A 的卷子丢了，B 和 C 可以把 A 的答案还原出来。这个"记对方答案"的过程就是校验。

特性	说明
最少硬盘数	3 块
可用容量	$(n-1) \times \text{单盘容量}$
读写性能	读取快，写入略慢（需计算校验）
容错能力	可坏 1 块盘
适用场景	企业文件服务器、Web 服务器

RAID 10：土豪方案（RAID 1+0）

工作原理：先做 RAID 1 镜像，再把镜像组做 RAID 0 条带化。兼顾速度和安全。

生活类比：你有 4 个保险柜，先两两配对互为备份（RAID 1），然后两组保险柜同时存取（RAID 0）。

特性	说明
最少硬盘数	4 块
可用容量	总容量的 50%
读写性能	非常优秀
容错能力	每组可坏 1 块
适用场景	数据库服务器、高性能计算

1.4 RAID 级别对比总结

级别	最少盘数	可用容量	容错	读性能	写性能
RAID 0	2	100%	无	☆☆☆	☆☆☆
RAID 1	2	50%	1块	☆☆	☆
RAID 5	3	(n-1)/n	1块	☆☆☆	☆☆
RAID 10	4	50%	每组1块	☆☆☆	☆☆☆

1.5 硬件 RAID vs 软件 RAID

对比项	硬件 RAID	软件 RAID
实现方式	专用 RAID 卡	操作系统实现
成本	高（需购买RAID卡）	低（免费）
性能	更好（独立处理器）	占用 CPU 资源
配置难度	需进 BIOS 配置	命令行操作
典型代表	LSI、Dell PERC	Linux mdadm

本课程重点讲解 Linux 软件 RAID（mdadm），因为它免费、灵活，在云计算环境中广泛使用。

第二部分：实践篇

2.1 实验环境准备

实验环境要求

项目	要求
操作系统	CentOS 7 / Rocky Linux 8 / Ubuntu 20.04+
硬盘数量	至少 4 块额外硬盘（可用虚拟机添加虚拟磁盘）
硬盘大小	每块 1GB 以上即可（实验用）
所需工具	mdadm

添加虚拟硬盘（以 VMware 为例）

步骤：

1. 关闭虚拟机

2. 编辑虚拟机设置 → 添加 → 硬盘

3. 选择 SCSI → 创建新虚拟磁盘

4. 大小设为 1GB

5. 重复以上步骤，添加 4 块硬盘

6. 启动虚拟机

确认硬盘已识别

查看所有块设备

lsblk

预期输出类似：

#	NAME	MAJ:MIN	RM	SIZE	RO	TYPE	MOUNTPPOINT
#	sda	8:0	0	20G	0	disk	
#	_sda1	8:1	0	1G	0	part	/boot
#	_sda2	8:2	0	19G	0	part	
#	sdb	8:16	0	1G	0	disk	← 新硬盘
#	sdc	8:32	0	1G	0	disk	← 新硬盘
#	sdd	8:48	0	1G	0	disk	← 新硬盘
#	sde	8:64	0	1G	0	disk	← 新硬盘

安装 mdadm 工具

```
# CentOS/RHEL
yum install -y mdadm

# Ubuntu/Debian
apt install -y mdadm
```

2.2 实验一：创建 RAID 0（条带化）

目标：使用 2 块硬盘创建 RAID 0，体验容量叠加

第一步：确认使用的硬盘

```
# 我们使用 /dev/sdb 和 /dev/sdc
lsblk /dev/sdb /dev/sdc
```

第二步：创建 RAID 0

```
# 创建 RAID 0 阵列
# -C：创建
# -v：显示详细信息
# /dev/md0：RAID 设备名
# -l 0：RAID 级别 0
# -n 2：使用 2 块硬盘
mdadm -Cv /dev/md0 -l 0 -n 2 /dev/sdb /dev/sdc
```

预期输出：

```
mdadm: chunk size defaults to 512K
mdadm: Defaulting to version 1.2 metadata
mdadm: array /dev/md0 started.
```

第三步：查看 RAID 状态

```
# 查看 RAID 详细信息
mdadm -D /dev/md0

# 或查看简要状态
cat /proc/mdstat
```

预期输出：

```
/dev/md0:
    Version : 1.2
  Raid Level : raid0
  Array Size : 2093056 (2044.00 MiB 2143.29 MB) ← 两块盘容量相加
  Raid Devices : 2
  Total Devices : 2
    ...
   Number    Major   Minor   RaidDevice State
     0         8      16         0     active sync  /dev/sdb
     1         8      32         1     active sync  /dev/sdc
```

第四步：格式化并挂载

```
# 格式化为 xfs 文件系统
mkfs.xfs /dev/md0

# 创建挂载点
mkdir -p /mnt/raid0

# 挂载
mount /dev/md0 /mnt/raid0

# 验证
df -h /mnt/raid0
```

第五步：测试读写

```
# 写入测试文件
echo "Hello RAID 0" > /mnt/raid0/test.txt

# 读取验证
cat /mnt/raid0/test.txt
```

第六步：清理实验环境

```
# 卸载
umount /mnt/raid0

# 停止 RAID
mdadm --stop /dev/md0

# 清除超级块
mdadm --zero-superblock /dev/sdb /dev/sdc
```

2.3 实验二：创建 RAID 1（镜像）

目标：使用 2 块硬盘创建 RAID 1，体验数据镜像和故障恢复

第一步：创建 RAID 1

```
mdadm -Cv /dev/md1 -l 1 -n 2 /dev/sdb /dev/sdc
```

第二步：查看同步进度

RAID 1 创建后会自动同步数据，这需要一点时间。


```
# 观察同步进度
watch -n 1 cat /proc/mdstat

# 输出示例:
# md1 : active raid1 sdc[1] sdb[0]
#      1046528 blocks super 1.2 [2/2] [UU]
#      [=====>.....] resync = 42.3% (443392/1046528)
```

[UU] 表示两块盘都正常，如果显示 [U_] 表示有一块盘故障。

第三步：格式化并使用

```
mkfs.xfs /dev/md1
mkdir -p /mnt/raid1
mount /dev/md1 /mnt/raid1

# 写入测试数据
echo "重要数据 - RAID 1 保护" > /mnt/raid1/important.txt
```

第四步：模拟硬盘故障

```
# 标记 /dev/sdc 为故障盘
mdadm /dev/md1 --fail /dev/sdc

# 查看状态
mdadm -D /dev/md1
```

关键输出：

State :	clean, degraded	← 降级状态
Number	Major	Minor
0	8	16
-	0	0
1	8	32

RaidDevice	State	
0	active sync	/dev/sdb
1	removed	
-	faulty	/dev/sdc ← 故障盘

第五步：验证数据仍可访问

```
# 即使一块盘故障，数据依然可读
cat /mnt/raid1/important.txt

# 输出：重要数据 - RAID 1 保护
```

第六步：移除故障盘并添加新盘

```
# 移除故障盘
mdadm /dev/md1 --remove /dev/sdc

# 假设 /dev/sdd 是新盘，添加到阵列
mdadm /dev/md1 --add /dev/sdd

# 观察重建过程
watch -n 1 cat /proc/mdstat
```

第七步：清理

```
umount /mnt/raid1
mdadm --stop /dev/md1
mdadm --zero-superblock /dev/sdb /dev/sdc /dev/sdd
```

2.4 实验三：创建 RAID 5

目标：使用 3 块硬盘创建 RAID 5，理解校验分布

第一步：创建 RAID 5

```
mdadm -Cv /dev/md5 -l 5 -n 3 /dev/sdb /dev/sdc /dev/sdd
```

第二步：观察同步

```
cat /proc/mdstat
```

输出示例：

```
# md5 : active raid5 sdd[3] sdc[1] sdb[0]
#      2093056 blocks super 1.2 level 5, 512k chunk, algorithm 2
[3/3] [UUU]
```

[UUU] 表示三块盘都正常。

第三步：验证容量

```
mdadm -D /dev/md5 | grep "Array Size"
```

输出示例：

```
# Array Size : 2093056 (2044.00 MiB)
# 3块1G的盘，可用约2G（损失1块盘容量用于校验）
```

第四步：格式化使用

```
mkfs.xfs /dev/md5
mkdir -p /mnt/raid5
mount /dev/md5 /mnt/raid5
```

写入数据

```
for i in {1..5}; do
    echo "测试文件 $i" > /mnt/raid5/file$i.txt
done
```

验证

```
ls -l /mnt/raid5/
```

第五步：模拟故障与恢复

```
# 模拟 /dev/sdc 故障
mdadm /dev/md5 --fail /dev/sdc
mdadm /dev/md5 --remove /dev/sdc

# 检查状态 - 降级但数据可用
mdadm -D /dev/md5 | grep State
cat /mnt/raid5/file1.txt

# 添加热备盘恢复
mdadm /dev/md5 --add /dev/sde

# 观察重建
watch -n 1 cat /proc/mdstat
```

第六步：清理

```
umount /mnt/raid5
mdadm --stop /dev/md5
mdadm --zero-superblock /dev/sdb /dev/sdc /dev/sdd /dev/sde
```

2.5 实验四：创建 RAID 10

目标：使用 4 块硬盘创建 RAID 10

创建 RAID 10

```
# RAID 10 需要偶数块盘，最少 4 块
mdadm -Cv /dev/md10 -l 10 -n 4 /dev/sdb /dev/sdc /dev/sdd /dev/sde
```

查看状态

```
mdadm -D /dev/md10
```

```
# 关键输出:
```

```
#      Raid Level : raid10
```

```
#      Array Size : 2093056 (约2G - 4块1G盘的50%)
```

```
#      Raid Devices : 4
```

理解 RAID 10 结构

```
# 查看布局
```

```
mdadm -D /dev/md10 | grep -A 10 "Number"
```

```
# 输出类似:
```

#	Number	Major	Minor	RaidDevice	State			
#	0	8	16	0	active sync	set-A	/dev/sdb	
#	1	8	32	1	active sync	set-B	/dev/sdc	
#	2	8	48	2	active sync	set-A	/dev/sdd	
#	3	8	64	3	active sync	set-B	/dev/sde	

set-A 和 set-B 分别是两个镜像组。

2.6 配置 RAID 永久生效

默认情况下，重启后 RAID 不会自动激活。需要保存配置。

保存 RAID 配置

```
# 扫描并保存配置到 mdadm.conf
```

```
mdadm --detail --scan >> /etc/mdadm.conf
```

```
# CentOS/RHEL 可能是
```

```
mdadm --detail --scan >> /etc/mdadm/mdadm.conf
```

```
# 查看配置
```

```
cat /etc/mdadm.conf
```

配置开机自动挂载

```
# 获取 UUID
blkid /dev/md0

# 编辑 /etc/fstab, 添加一行
# UUID=xxx-xxx-xxx    /mnt/raid0    xfs    defaults    0 0

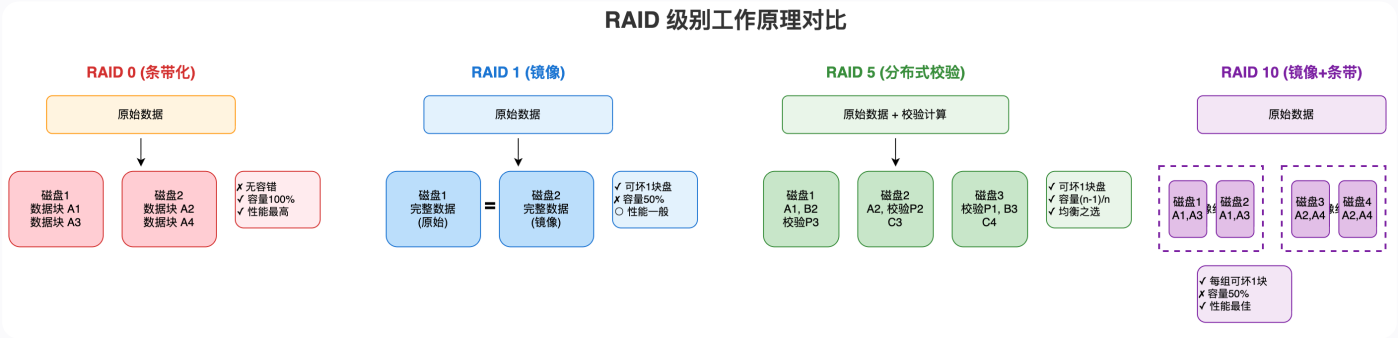
# 测试 fstab 配置
mount -a
```

2.7 常用 RAID 管理命令速查表

功能	命令
创建 RAID	mdadm -Cv /dev/mdX -l 级别 -n 盘数 设备列表
查看详情	mdadm -D /dev/mdX
查看状态	cat /proc/mdstat
标记故障	mdadm /dev/mdX --fail /dev/sdX
移除磁盘	mdadm /dev/mdX --remove /dev/sdX
添加磁盘	mdadm /dev/mdX --add /dev/sdX
停止 RAID	mdadm --stop /dev/mdX
清除超级块	mdadm --zero-superblock /dev/sdX
扫描阵列	mdadm --assemble --scan
保存配置	mdadm --detail --scan >> /etc/mdadm.conf

第三部分：配图

图1：RAID 级别工作原理对比图



第四部分：课后练习

练习一：基础操作

- 使用 4 块 1GB 的虚拟硬盘创建一个 RAID 5 阵列
- 格式化为 ext4 文件系统并挂载到 `/data`
- 创建 5 个测试文件
- 模拟一块硬盘故障，验证数据是否仍可访问
- 添加新硬盘恢复阵列
- 配置开机自动挂载

练习二：计算题

某公司有 6 块 4TB 硬盘，请计算以下 RAID 方案的可用容量：

RAID 级别	可用容量	计算过程
RAID 0	?	
RAID 1	?	
RAID 5	?	
RAID 10	?	

第五部分：课程总结

核心知识点回顾

RAID 是什么：把多块硬盘组合成一个逻辑磁盘，实现性能提升或数据冗余。

常用级别：RAID 0 追求速度但无保护，RAID 1 完全镜像最安全但浪费一半容量，RAID 5 是性能与安全的平衡之选，RAID 10 是土豪方案兼顾速度和安全。

核心命令：`mdadm -Cv` 创建阵列，`mdadm -D` 查看详情，`cat /proc/mdstat` 查看状态，`--fail/--remove/--add` 处理故障。

生产环境建议

系统盘推荐 RAID 1，保证系统能启动。数据库服务器推荐 RAID 10，性能和安全都有保障。文件服务器推荐 RAID 5 或 RAID 6，容量利用率高。临时数据或缓存可用 RAID 0，但千万别存重要数据。

无论使用什么 RAID 级别，都要定期备份，RAID 不能替代备份。